Практическая работа № 1

РАЗВЁРТЫВАНИЕ SCADA-СИСТЕМЫ И ЭМУЛЯЦИЯ ПЛК

Методические указания к выполнению

Стек: C++ (libmodbus) · Go · PostgreSQL · Docker · FUXA / MasterSCADA 4D

2026

Содержание

Введение **3**
1. Цель и задачи работы **3**
1.1. Что должно получиться в итоге 3
2. Что понадобится **5**
2.1. Компьютер и программы 5
2.2. Что нужно знать заранее 5
3. Краткие теоретические сведения **6**
3.1. SCADA и её место в АСУ ТП 6
3.2. Реальное время 6
3.3. Протокол Modbus 6
3.4. Теги, опрос и архив 7
3.5. Аварийная сигнализация 8
3.6. Объект управления 8
4. Устройство учебного стенда **9**
4.1. Состав 9
4.2. Карта регистров ПЛК 9
4.3. Репозиторий и задания в коде 10
Этап 0. Подготовка рабочего места 11
Этап 1. Эмулятор ПЛК (TODO 1–4) 11
Этап 2. База данных (TODO 5–7) 17
Этап 3. Шлюз (TODO 8–11) 20
Этап 4. Запуск стенда и автоматическая проверка 26
Этап 5Ф. Мнемосхема в FUXA 27
Этап 5М. MasterSCADA 4D (в учебном кабинете) 34
Этап 6. Эксперименты 35
6. Дополнительные задания **36**
6.1. Автоматический регулятор охладителя 36
6.2. Тренды и архив в Grafana 37
6.3. Уведомления через LISTEN/NOTIFY 37
6.4. Резервирование шлюза 38
6.5. Фильтрация измерений 38
7. Частые проблемы **38**
8. Сдача работы **39**
9. Контрольные вопросы **39**
10. Литература и ссылки **40**
Приложение А. Памятка по командам **41**

Введение

В этой работе вы соберёте небольшую, но полноценную систему диспетчерского управления. Внизу будет программный контроллер, который моделирует нагрев химического реактора и отдаёт данные по протоколу Modbus TCP. Над ним — шлюз, который опрашивает контроллер, складывает измерения в базу PostgreSQL и формирует аварийные события. Сверху — SCADA-система с мнемосхемой, на которой оператор видит температуру, управляет охладителем и получает сигнал об аварии.

Готовыми даны только вспомогательные части и тесты. Основное вы пишете сами: сервер Modbus и модель объекта на C++ (около 120 строк), схему базы на SQL (около 70 строк) и шлюз на Go (около 300 строк). Для каждого модуля в указаниях есть пошаговый алгоритм и справочник по функциям используемых библиотек с примерами. Всё, что вы сделаете, проверяется автоматически, поэтому вы сразу видите, работает решение или нет.

Работа рассчитана примерно на 8 академических часов. Если вы раньше не сталкивались с Docker, заложите ещё час на подготовку рабочего места.

1. Цель и задачи работы

Цель работы — разобраться, как устроен канал «контроллер → SCADA → архив», и получить практические навыки работы с протоколом Modbus TCP, базой данных PostgreSQL и языками C++ и Go в задачах промышленной автоматизации.

Задачи:

1) написать на C++ эмулятор ПЛК: сервер Modbus TCP, обслуживающий несколько клиентов, и математическую модель объекта управления;
2) проверить обмен с контроллером утилитой mbpoll;
3) спроектировать схему базы данных: архив, уставки, журнал событий, представление текущих значений и уведомления;
4) написать на Go шлюз: драйвер ПЛК, работу с базой, алгоритм аварийной сигнализации с гистерезисом и цикл опроса;
5) развернуть стенд в Docker и пройти автоматическую проверку;
6) построить мнемосхему в SCADA-системе FUXA (или MasterSCADA 4D в учебном кабинете), настроить управление и аварии;
7) провести эксперименты со стендом и объяснить их результаты в отчёте.

1.1. Что должно получиться в итоге

Объект управления — химический реактор R-1 периодического действия. Его настоящего у нас нет: поведение реактора воспроизводит эмулятор ПЛК. Для

SCADA разницы нет никакой, она общается с эмулятором по тому же протоколу и через те же регистры, что и с реальным контроллером.

На реакторе установлены:

Оборудование	Тип	Что делает	Сигнал в ПЛК
электронагреватель	исполнительный механизм	нагревает содержимое реактора, мощность задаётся от 0 до 100 %	задание — Holding Register 0, фактическая мощность — Input Register 1
охлаждающая рубашка с клапаном	исполнительный механизм	при открытом клапане через рубашку идёт хладагент с температурой 10 °C	команда — Coil 0
датчик температуры (термопара)	аналоговый датчик	измеряет температуру внутри реактора с шагом 0,1 °C, показания немного шумят	Input Register 0 (значение × 10)
датчик загазованности	дискретный датчик	срабатывает при утечке горючего газа в помещении	Discrete Input 0

Нормальная рабочая температура — до 80 °C. Выше 80 °C технолог должен вмешаться (предупредительная сигнализация H), выше 90 °C — аварийная ситуация (HH). Срабатывание датчика газа — всегда авария. Утечку в стенде имитирует преподаватель отдельной катушкой (Coil 1), оператор её с мнемосхемы устранить не может.

Если ничего не делать, при мощности 50 % реактор за полторы минуты нагреется выше 80 °C и продолжит греться до 145 °C. Включённое охлаждение удерживает температуру около 38 °C. Задача оператора — следить за температурой и вовремя включать охлаждение или снижать мощность.

К концу работы у вас будет действующая система диспетчерского управления этим реактором:

- эмулятор ПЛК на C++, который моделирует реактор и отвечает по Modbus TCP нескольким клиентам одновременно;
- база данных PostgreSQL с архивом всех измерений, уставками и журналом аварийных событий;
- шлюз на Go, который раз в секунду опрашивает ПЛК, пишет архив и формирует события при выходе температуры за уставки и при срабатывании датчика газа;
- мнемосхема в FUXA (или MasterSCADA 4D), на которой оператор видит температуру, мощность и состояние охлаждения, может включить и выключить охладитель, задать мощность нагревателя, получает сигнал об аварии и квитирует его.

Всё это запускается одной командой `docker compose up` и проходит автоматическую проверку. Как примерно выглядит экран оператора, показано на рисунке в конце этапа 5Ф.

Сдаёте вы три вещи: репозиторий с кодом и проектом FUXA (с зелёной отметкой CI), отчёт с результатами экспериментов и ответами на контрольные вопросы и устную защиту, на которой нужно показать работающий стенд и объяснить свой код.

2. Что понадобится

2.1. Компьютер и программы

Стенд проверен на Ubuntu 26.04 с Docker 29.1 и Docker Compose 2.40. Подойдёт любой современный дистрибутив Linux. Работу в Windows через WSL2 и на macOS мы не проверяли; скорее всего, она возможна, но решать проблемы окружения придётся самостоятельно.

Нужно примерно 4 ГБ свободной оперативной памяти и 5 ГБ места на диске. Для первой сборки нужен интернет: Docker скачивает образы, Go — зависимости, FUXA — драйвер Modbus.

Программа	**Зачем**	**Пакет в Ubuntu**
Docker Engine и Docker Compose v2	запуск стенда	docker.io, docker-compose-v2
git	получение кода	git
g++ (C++17), CMake, pkg-config, libmodbus	сборка эмулятора без Docker	build-essential, cmake, pkg-config, libmodbus-dev
Go 1.26 или новее	сборка и тесты шлюза	golang (или архив с go.dev)
mbpoll	ручной опрос контроллера	mbpoll
psql	запросы к базе	postgresql-client
браузер	работа с FUXA	любой

Установка одной командой:

```
sudo apt install docker.io docker-compose-v2 git build-essential cmake pkg-config \
    libmodbus-dev golang mbpoll postgresql-client
sudo usermod -aG docker $USER
```

После `usermod` выйдите из сеанса и войдите снова, иначе Docker будет требовать `sudo`. Проверьте версию Go командой `go version`: если пакет из репозитория старше 1.26, поставьте Go с сайта go.dev.

2.2. Что нужно знать заранее

Хватит базового C++ (функции, указатели, приведение типов), базового Go (структуры, срезы, обработка ошибок), SQL на уровне SELECT/INSERT и умения работать в терминале Linux. Опыт с Docker желателен, но не обязателен: все нужные команды приведены в тексте.

3. Краткие теоретические сведения

3.1. SCADA и её место в АСУ ТП

SCADA (Supervisory Control And Data Acquisition — диспетчерское управление и сбор данных) — это программный комплекс верхнего уровня автоматизированной системы управления технологическим процессом. Принято выделять несколько уровней АСУ ТП:

- полевой уровень — датчики и исполнительные механизмы: термопары, расходомеры, клапаны, двигатели;
- уровень управления — программируемые логические контроллеры (ПЛК), которые циклически читают входы, выполняют программу и выставляют выходы;
- диспетчерский уровень — SCADA: мнемосхемы, тревоги, архив, команды оператора;
- уровень предприятия — MES и ERP, которые берут из SCADA сводные данные.

Основные функции SCADA: сбор данных с контроллеров, отображение их оператору, аварийная и предупредительная сигнализация, архивирование, передача команд оператора и построение отчётов. Важно понимать, что SCADA не заменяет контроллер. Если связь между ними пропала, ПЛК продолжает управлять процессом по своей программе, а оператор просто временно теряет обзор.

3.2. Реальное время

В системах реального времени важна не только правильность результата, но и момент, к которому он получен. Различают жёсткое реальное время, где пропуск срока недопустим (защиты, регуляторы в ПЛК с циклом единицы миллисекунд), и мягкое, где редкая задержка ухудшает качество, но не приводит к аварии.

SCADA работает в мягком реальном времени. Опрос раз в 0,5–2 секунды — обычная практика, а за быстрые и ответственные действия отвечает контроллер. В нашем стенде шлюз опрашивает ПЛК раз в секунду, FUXA — тоже раз в секунду, независимо от шлюза.

3.3. Протокол Modbus

Modbus появился в 1979 году у компании Modicon и до сих пор остаётся самым распространённым промышленным протоколом благодаря простоте. Обмен строится по схеме «запрос — ответ»: клиент (master) спрашивает, сервер (slave) отвечает. Сам сервер никогда не начинает обмен. Есть последовательный вариант Modbus RTU (обычно поверх RS-485) и Modbus TCP поверх Ethernet. Стандартный порт Modbus TCP — 502. В стенде используется порт 5020: номера ниже 1024 в Linux требуют прав администратора, а эмулятор удобно запускать от обычного пользователя.

Данные сервера разложены по четырём областям:

Область	Размер элемента	Доступ	Функции чтения / записи	Пример
Coils (катушки)	1 бит	чтение и запись	01 / 05, 15	команда «включить насос»
Discrete Inputs (дискретные входы)	1 бит	только чтение	02 / —	концевой выключатель
Input Registers (входные регистры)	16 бит	только чтение	04 / —	измеренная температура
Holding Registers (регистры хранения)	16 бит	чтение и запись	03 / 06, 16	уставка, задание

Регистр хранит 16 бит без знака. Отрицательные числа передают в дополнительном коде (int16), дробные — масштабированными целыми: например, температуру 38,4 °C контроллер кладёт в регистр как 384, а SCADA делит на 10. Для 32-битных и вещественных значений занимают два соседних регистра, и тут легко ошибиться с порядком слов.

Отдельно о нумерации. В самом протоколе адреса начинаются с нуля. В документации же исторически пишут номера с единицы и с префиксом области: 000001 — первая катушка, 100001 — первый дискретный вход, 300001 — первый входной регистр, 400001 — первый регистр хранения. Утилита mbpoll и FUXA ждут номер с единицы, а библиотеки libmodbus и simonvetter/modbus — адрес с нуля. Это классический источник ошибки «на единицу», и в работе вы с ним столкнётесь.

В запросе Modbus TCP есть ещё поле Unit ID (идентификатор устройства). Оно пришло из RTU, где на одной линии висит несколько устройств. Наш эмулятор отвечает на Unit ID 1.

В классическом Modbus нет ни аутентификации, ни шифрования: любой, кто достучался до порта, может прочитать и записать любые разрешённые регистры. Поэтому промышленные сети отделяют от офисных, а доступ к ПЛК ограничивают межсетевыми экранами. Существует спецификация Modbus/TCP Security с TLS (порт 802), но в действующем оборудовании она встречается редко.

3.4. Теги, опрос и архив

Тег — это именованная переменная SCADA, связанная с конкретным адресом в контроллере: «reactor_temp — входной регистр 0, масштаб 0,1, °C». Список тегов хранится в конфигурации. В нашем стенде это таблица `tags` в базе данных.

Шлюз периодически опрашивает контроллер и записывает каждое значение с меткой времени в таблицу `measurements`. Такие данные называют временными

рядами. Они растут быстро: 5 тегов раз в секунду — это 432 000 строк в сутки. Поэтому для таблицы сделан составной индекс по (tag_id, time DESC), а последние значения удобно получать запросом с `DISTINCT ON`. В промышленных системах для архива используют специализированные хранилища (историаны) или расширения PostgreSQL вроде TimescaleDB; для учебной задачи хватает обычной таблицы.

3.5. Аварийная сигнализация

Для аналогового сигнала задают уставки — пороговые значения. Обычно их четыре: LL (low-low, аварийно низкое), L (предупредительно низкое), H (предупредительно высокое) и HH (high-high, аварийно высокое). У нас для температуры реактора H = 80 °C и HH = 90 °C.

Если сравнивать значение с уставкой «в лоб», при колебаниях около порога (а измерение всегда шумит) авария будет возникать и сниматься десятки раз в минуту. Это называют дребезгом, и он мешает оператору больше, чем помогает. Поэтому вводят гистерезис, он же зона возврата (deadband): авария возникает при достижении уставки, а снимается только после того, как значение ушло обратно дальше чем на ширину зоны. При H = 80 и зоне 2 °C предупреждение появится на 80,0 и исчезнет только ниже 78,0.

Каждая смена состояния записывается в журнал событий с временем, значением и текстом сообщения. Оператор квитирует (подтверждает) аварию, тем самым показывая, что увидел её. В стенде квитирование есть в FUXA, а в базе для него предусмотрено поле `acked_at`.

3.6. Объект управления

Эмулятор моделирует реактор с электронагревателем и охлаждающей рубашкой. Температура описывается уравнением теплового баланса первого порядка:

```
dT/dt = A·P − B·(T − T_amb) − C·cooler·(T − T_cool)
```

где T — температура, °C; P — мощность нагревателя, % (0–100); cooler — 1, если охладитель включён, иначе 0; A = 0,02 °C/(с·%); B = 0,008 1/с — теплообмен с окружающей средой; C = 0,03 1/с — интенсивность охлаждения; T_amb = 20 °C; T_cool = 10 °C.

В установившемся режиме производная равна нулю, откуда можно найти конечную температуру. Без охлаждения при P = 50 %:

```
T_уст = T_amb + A·P / B = 20 + 0,02·50 / 0,008 = 145 °C
```

С охладителем T_уст = (A·P + B·T_amb + C·T_cool) / (B + C) ≈ 38,4 °C. Постоянная времени без охлаждения τ = 1/B = 125 с, поэтому уставку H = 80 °C реактор проходит примерно через 80 секунд после старта.

Компьютер не умеет решать дифференциальные уравнения непрерывно, поэтому в эмуляторе используется явный метод Эйлера: каждые DT = 0,1 с к температуре прибавляется производная, умноженная на шаг:

```
T(k+1) = T(k) + h · f(T(k)),   h = DT · PLC_SPEED
```

Параметр `PLC_SPEED` ускоряет модельное время. При PLC_SPEED = 10 за одну реальную секунду проходит 10 модельных, и все процессы идут в 10 раз быстрее. Это удобно для отладки, но точность метода с ростом шага падает.

4. Устройство учебного стенда

4.1. Состав

Стенд запускается одной командой `docker compose up` и состоит из четырёх контейнеров:

Сервис	Что это	Порт на компьютере
plc	эмулятор ПЛК на C++ (libmodbus)	5020 (Modbus TCP)
db	PostgreSQL 17	5432
gateway	шлюз на Go: опрос ПЛК, архив, аварии	—
fuxa	SCADA-система FUXA 1.3.5	1881 (веб-интерфейс)

```
plc (C++) --- Modbus TCP :5020 ---> gateway (Go) ---> PostgreSQL
    |
    +------ Modbus TCP :5020 ---> FUXA (мнемосхема, аварии)
```

Обратите внимание: шлюз и FUXA опрашивают контроллер независимо друг от друга. Это обычная ситуация: к одному ПЛК часто подключены и SCADA, и система архивирования, и инженерная станция. Внутри Docker контейнеры находят друг друга по именам сервисов, поэтому шлюз обращается к контроллеру по адресу `plc:5020`, а не `localhost:5020`.

4.2. Карта регистров ПЛК

Область	Адрес (с 0)	Номер в mbpoll и FUXA (с 1)	Назначение
Input Register	0	1 (300001)	температура ×10, int16
Input Register	1	2 (300002)	фактическая мощность нагревателя, %
Holding Register	0	1 (400001)	задание мощности нагревателя, % (по умолчанию 50)
Holding Register	1	2 (400002)	амплитуда шума измерения ×10
Coil	0	1 (000001)	охладитель
Coil	1	2 (000002)	имитация утечки газа (для преподавателя)
Discrete Input	0	1 (100001)	датчик загазованности

Датчик газа повторяет состояние катушки 1. Так в стенде моделируется событие, которое оператор не может «выключить» с мнемосхемы: он видит аварию, но устранить её причину может только тот, кто управляет имитацией.

4.3. Репозиторий и задания в коде

Код лежит в публичном репозитории-шаблоне https://github.com/Chebupelinka/scada-lab-student. Готовыми в нём даны только вспомогательные части: разбор параметров командной строки эмулятора, таблица `tags`, типы данных шлюза (`gateway/types.go`), тесты, файл docker-compose.yml и скрипт проверки. Всё остальное вы пишете сами. Места для кода помечены комментарием `TODO N`, номера идут в том порядке, в котором их удобно выполнять:

№	Файл	Что написать	Объём, строк	Чем проверяется
1	plc-emulator/src/main.cpp	create_mapping: карта регистров	8–10	mbpoll
2	plc-emulator/src/main.cpp	simulate: модель реактора	30–40	mbpoll, check.sh
3	plc-emulator/src/main.cpp	serve: сервер Modbus TCP на select()	35–45	mbpoll, check.sh
4	plc-emulator/src/main.cpp	main: запуск и остановка	25–35	mbpoll
5	db/init/01_schema.sql	таблицы measurements, alarm_limits, events, commands	30–40	check.sh, store_test.go
6	db/init/01_schema.sql	представление current_values, функция и триггер NOTIFY	20–25	check.sh, store_test.go
7	db/init/02_data.sql	теги и уставки	10–12	check.sh
8	gateway/plc.go	драйвер ПЛК на simonvetter/modbus	60–75	plc_test.go
9	gateway/store.go	хранилище на pgx	80–100	store_test.go
10	gateway/alarms.go	Evaluate и Message	50–55	alarms_test.go
11	gateway/main.go	настройки, цикл опроса, аварии	90–110	check.sh

Всего около 450–550 строк. Объём указан для ориентира: если у вас вышло заметно больше, скорее всего, где-то есть лишнее.

Сигнатуры функций в шаблонах менять нельзя: их вызывают тесты и остальной код. Внутреннее устройство (поля структур, вспомогательные функции) вы выбираете сами.

Тесты лежат в каталоге `gateway` рядом с кодом: `alarms_test.go` проверяет логику аварий и работает без стенда, `plc_test.go` и `store_test.go` подключаются

к работающему эмулятору и базе. Если переменные окружения `MODBUS_URL` и `DATABASE_URL` не заданы, интеграционные тесты пропускаются. Прочитайте тесты до того, как писать код: по ним видно, чего ждут от каждой функции.

Этап 0. Подготовка рабочего места

1) Установите программы из раздела 2.1 и проверьте, что Docker работает без sudo:

```
docker run --rm hello-world
```

2) Создайте свою копию репозитория. Войдите в GitHub, откройте https://github.com/Chebupelinka/scada-lab-student, нажмите Use this template → Create a new repository, задайте имя (например, scada-lab-ivanov) и видимость Private. В настройках своего репозитория (Settings → Collaborators) добавьте преподавателя.

Если аккаунта GitHub нет, работать можно и без него: склонируйте шаблон напрямую. Но тогда работу придётся сдавать вживую, без автоматической проверки.

3) Склонируйте свою копию и перейдите в неё:

```
git clone https://github.com/<ваш-логин>/scada-lab-ivanov.git
cd scada-lab-ivanov
cp .env.example .env
```

Файл `.env` задаёт параметры стенда: пароли, порты, ускорение модели. На время отладки удобно поставить в нём `PLC_SPEED=10`.

4) Заранее скачайте образы, чтобы потом не ждать:

```
docker compose pull
```

Команда скачает PostgreSQL и FUXA. Образы plc и gateway собираются из исходников позже.

Этап 1. Эмулятор ПЛК (TODO 1–4)

Что дано и что нужно сделать

Откройте `plc-emulator/src/main.cpp`. В начале файла объявлены номера регистров (`IR_TEMPERATURE`, `HR_HEATER_SETPOINT` и т. д.), константы модели (`A`, `B`, `C`, `T_AMB`, `T_COOL`, `DT`), флаг остановки `g_running`, обработчик сигнала `on_signal` и структура `Config` с функцией `parse_config`, которая читает порт, ускорение и начальную мощность из переменных окружения и ключей командной строки. Ниже стоят четыре пустые функции, их и нужно написать.

Программа работает в двух потоках. Поток модели (`simulate`) каждые 0,1 с пересчитывает температуру и кладёт результат в карту регистров. Главный поток (`serve`) принимает подключения клиентов и отвечает на их запросы, читая и

записывая ту же карту. Чтобы потоки не мешали друг другу, любое обращение к карте выполняется под одним мьютексом.

Справочник: libmodbus

Библиотека libmodbus (заголовок `<modbus/modbus.h>`) умеет быть и клиентом, и сервером Modbus. Для сервера нужны следующие функции.

Функция	Параметры	Возвращает
`modbus_mapping_new(nb_bits, nb_input_bits, nb_registers, nb_input_registers)`	количество катушек, дискретных входов, регистров хранения и входных регистров	указатель на `modbus_mapping_t` или NULL при ошибке
`modbus_mapping_free(map)`	карта регистров	—
`modbus_new_tcp(ip, port)`	адрес и порт; для сервера ip = nullptr — слушать все адреса	указатель на контекст `modbus_t` или NULL
`modbus_tcp_listen(ctx, nb_connection)`	контекст; длина очереди ожидающих подключений	дескриптор слушающего сокета или −1
`modbus_set_socket(ctx, s)`	контекст; сокет клиента, с которым будет идти обмен	0 или −1
`modbus_receive(ctx, req)`	контекст; буфер размером `MODBUS_TCP_MAX_ADU_LENGTH`	длина запроса; 0 — запрос нужно пропустить; −1 — ошибка или клиент отключился
`modbus_reply(ctx, req, req_length, map)`	контекст; запрос и его длина из `modbus_receive`; карта	число отправленных байт или −1
`modbus_strerror(errnum)`	код ошибки, обычно `errno`	текст ошибки
`modbus_free(ctx)`	контекст	—

Структура `modbus_mapping_t` содержит четыре массива, индекс в которых совпадает с адресом Modbus (с нуля): `tab_bits` и `tab_input_bits` типа `uint8_t*` (0 или 1), `tab_registers` и `tab_input_registers` типа `uint16_t*`. Функция `modbus_reply` сама разбирает запрос: читает из нужного массива, пишет в `tab_bits` и `tab_registers`, а на запрос к несуществующему адресу отвечает ошибкой ILLEGAL DATA ADDRESS. Запись во входные регистры и дискретные входы клиент сделать не может, их меняет только ваш код.

Пример: карта на 4 катушки и 10 регистров хранения, начальное значение регистра 3 и чтение катушки 0.

```
modbus_mapping_t* m = modbus_mapping_new(4, 0, 10, 0);
if (m == nullptr) {
    std::fprintf(stderr, "ошибка: %s\n", modbus_strerror(errno));
    return 1;
}
m->tab_registers[3] = 1500;
bool pump_on = m->tab_bits[0] != 0;
```

Пример: сервер на порту 1502, который принимает одного клиента и обрабатывает один запрос. В вашей программе вместо этого будет цикл на `select()`.

```
modbus_t* ctx = modbus_new_tcp(nullptr, 1502);
int listen_fd = modbus_tcp_listen(ctx, 1);
int client = accept(listen_fd, nullptr, nullptr);
modbus_set_socket(ctx, client);
uint8_t req[MODBUS_TCP_MAX_ADU_LENGTH];
int len = modbus_receive(ctx, req);
if (len > 0) modbus_reply(ctx, req, len, m);
close(client);
```

Справочник: select() и сокеты

Функция `select()` из `<sys/select.h>` ждёт, пока на одном из сокетов появятся данные. Это позволяет одному потоку обслуживать много клиентов без отдельного потока на каждого.

Вызов	Назначение
`fd_set set; FD_ZERO(&set);`	создать пустое множество дескрипторов
`FD_SET(fd, &set)` / `FD_CLR(fd, &set)`	добавить / убрать дескриптор
`FD_ISSET(fd, &set)`	проверить, есть ли дескриптор в множестве
`select(nfds, &read_set, nullptr, nullptr, &timeout)`	ждать готовности; nfds — наибольший дескриптор + 1; возвращает число готовых дескрипторов, 0 по таймауту, −1 при ошибке
`accept(listen_fd, nullptr, nullptr)`	принять подключение; возвращает дескриптор клиента
`close(fd)`	закрыть сокет

`select()` изменяет переданное множество: после вызова в нём остаются только готовые дескрипторы. Поэтому общее множество клиентов храните отдельно и перед каждым вызовом копируйте. Таймаут задаётся структурой `timeval{секунды, микросекунды}`. Без таймаута программа не заметит, что `g_running` стал false, пока не придёт очередной запрос. Если `select()` вернул −1 и `errno == EINTR`, вызов прервал сигнал, это не ошибка.

Пример: ждать данных на двух сокетах не дольше 200 мс.

```
fd_set all;
FD_ZERO(&all);
FD_SET(a, &all);
FD_SET(b, &all);
int maxfd = std::max(a, b);

fd_set ready = all;
timeval tv{0, 200000};
int n = select(maxfd + 1, &ready, nullptr, nullptr, &tv);
if (n > 0 && FD_ISSET(a, &ready)) { /* читать из a */ }
```

Справочник: потоки и прочее из стандартной библиотеки

Конструкция	Назначение
`std::thread th(f, arg1, arg2)`	запустить функцию f в новом потоке; ссылки передают через `std::ref(x)` и `std::cref(x)`
`th.join()`	дождаться завершения потока
`std::mutex m;`	мьютекс
`std::lock_guard<std::mutex> lock(m);`	захватить мьютекс до конца блока `{ }`
`std::atomic<bool>`	флаг, который безопасно читать и писать из разных потоков
`std::signal(SIGINT, handler)`	вызвать handler по Ctrl+C; `SIG_IGN` вместо handler — игнорировать сигнал

Конструкция	Назначение
`std::this_thread::sleep_for(std::chrono::duration<double>(0.1))`	пауза 0,1 с
`std::mt19937 rng(std::random_device{}())`	генератор случайных чисел
`std::uniform_real_distribution<double> d(-1.0, 1.0); d(rng)`	случайное число от −1 до 1
`std::clamp(x, lo, hi)`	ограничить x диапазоном
`std::lround(x)`	округлить до ближайшего целого (long)

Пример: поток, который раз в секунду увеличивает общий счётчик, пока не сброшен флаг.

```
std::atomic<bool> running{true};
std::mutex mtx;
int counter = 0;

std::thread worker([&] {
    while (running) {
        { std::lock_guard<std::mutex> lock(mtx); ++counter; }
        std::this_thread::sleep_for(std::chrono::seconds(1));
    }
});
// ...
running = false;
worker.join();
```

TODO 1. create_mapping

1) Создайте карту функцией `modbus_mapping_new`: 2 катушки, 1 дискретный вход, 2 регистра хранения, 2 входных регистра (см. карту в разделе 4.2).
2) Если вернулся nullptr, верните nullptr.
3) Запишите начальные значения: в регистр `HR_HEATER_SETPOINT` — `cfg.heater_default`, в регистр `IR_TEMPERATURE` — `T_AMB · 10`.
4) Верните указатель на карту.

TODO 2. simulate

Функция выполняется в отдельном потоке, пока `g_running` равен true.

1) Перед циклом заведите переменную температуры `t = T_AMB`, генератор случайных чисел и распределение от −1 до 1.
2) В начале каждой итерации под мьютексом прочитайте из карты: задание мощности (`HR_HEATER_SETPOINT`, ограничьте 0–100 через `std::clamp`), амплитуду шума (`HR_NOISE`, делённую на 10), состояние охладителя (`COIL_COOLER`) и имитации утечки (`COIL_GAS_LEAK`). Держите мьютекс как можно меньше: только на чтение.
3) Вне мьютекса сделайте шаг метода Эйлера (раздел 3.6): `h = DT · cfg.speed`, `t += h · (A·P − B·(t − T_AMB) − C·cooler·(t − T_COOL))`.
4) Посчитайте измеренное значение: `t` плюс шум, умноженный на случайное число от −1 до 1.

5) Под мьютексом запишите: в `IR_TEMPERATURE` — измеренное значение × 10, округлённое и приведённое сначала к `int16_t`, затем к `uint16_t`; в `IR_HEATER_ACTUAL` — мощность; в `DI_GAS` — состояние утечки.

6) Раз в секунду (каждые 10 итераций) выведите строку состояния в формате `T= 29.6 °C heater= 50 % cooler=OFF gas=0` и вызовите `std::fflush(stdout)`: без этого в журнале Docker вывод появляется с задержкой.

7) Засните на `DT` секунд.

Подумайте, что произойдёт с отрицательной температурой, если пропустить промежуточное приведение к `int16_t`.

TODO 3. serve

1) Заведите множество клиентов `fd_set`, добавьте в него слушающий сокет `server`, запомните наибольший дескриптор.

2) Пока `g_running`: скопируйте множество, вызовите `select()` с таймаутом 200 мс. При −1 и `EINTR` продолжите цикл, при другой ошибке выведите `perror` и выйдите.

3) Переберите дескрипторы от 0 до наибольшего. Для каждого готового:

- если это `server` — примите подключение через `accept`, добавьте дескриптор клиента в множество, обновите наибольший, выведите «Клиент подключён (fd=N)»;
- иначе — `modbus_set_socket(ctx, fd)`, затем `modbus_receive`. Если вернулось положительное число, под мьютексом вызовите `modbus_reply`. Если −1 — закройте сокет, уберите его из множества и выведите «Клиент отключён (fd=N)».

4) После выхода из цикла закройте все клиентские сокеты.

TODO 4. main

1) Установите обработчики: `SIGINT` и `SIGTERM` — на `on_signal`, `SIGPIPE` — игнорировать. Без последнего программа завершится, если клиент отключится в момент отправки ответа.

2) Создайте карту (`create_mapping`), контекст `modbus_new_tcp(nullptr, cfg.port)` и слушающий сокет `modbus_tcp_listen(ctx, 16)`. На каждом шаге проверяйте результат: при ошибке выведите сообщение с `modbus_strerror(errno)` в stderr и верните 1.

3) Выведите строку `PLC emulator: Modbus TCP на порту 5020, ускорение x10.0`.

4) Создайте мьютекс и запустите поток `simulate`, передав карту, мьютекс по ссылке (`std::ref`) и конфигурацию (`std::cref`).

5) Вызовите `serve`. Функция вернётся, когда по сигналу сбросится `g_running`.

6) Выведите «Остановка...», дождитесь потока (`join`), закройте слушающий сокет, освободите контекст и карту. Верните 0.

Сборка и проверка

```
cmake -S plc-emulator -B plc-emulator/build
cmake --build plc-emulator/build
./plc-emulator/build/plc-emulator --speed 10
```

Раз в секунду эмулятор печатает состояние. Если всё написано верно, температура растёт:

```
PLC emulator: Modbus TCP на порту 5020, ускорение x10.0
T=  29.6 °C  heater= 50 %  cooler=OFF  gas=0
T=  38.6 °C  heater= 50 %  cooler=OFF  gas=0
T=  46.8 °C  heater= 50 %  cooler=OFF  gas=0
```

Не закрывая эмулятор, во втором терминале прочитайте два входных регистра (ключ `-t 3`, номера с 1, `-1` — опросить один раз):

```
mbpoll -m tcp -p 5020 -t 3 -r 1 -c 2 -1 localhost
```

В конце вывода должны быть строки вида:

```
-- Polling slave 1...
[1]:    394
[2]:    50
```

Число 394 — это 39,4 °C. Теперь поуправляйте объектом:

```
# включить охладитель (Coil 1)
mbpoll -m tcp -p 5020 -t 0 -r 1 -1 localhost 1
# задание мощности 0 % (Holding 1)
mbpoll -m tcp -p 5020 -t 4 -r 1 -1 localhost 0
# имитация утечки газа (Coil 2)
mbpoll -m tcp -p 5020 -t 0 -r 2 -1 localhost 1
# прочитать датчик газа (Discrete Input 1)
mbpoll -m tcp -p 5020 -t 1 -r 1 -c 1 -1 localhost
```

Убедитесь, что с охладителем температура останавливается около 38 °C, а при нулевой мощности падает. Проверьте, что сервер обслуживает несколько клиентов: запустите в двух терминалах непрерывный опрос (`mbpoll` без ключа `-1`), оба должны получать ответы, а эмулятор — печатать «Клиент подключён» для каждого. Попробуйте записать что-нибудь во входной регистр (`-t 3 ... localhost 5`). mbpoll откажется сразу: `Unable to write read-only element`. Причина в самом протоколе: функции записи входных регистров в Modbus просто нет.

Остановите эмулятор (Ctrl+C) и убедитесь, что он вывел «Остановка...» и завершился. Если этого не сделать, на следующих этапах контейнер plc не сможет занять порт 5020.

Этап 2. База данных (TODO 5–7)

Что дано и что нужно сделать

В `db/init/01_schema.sql` уже есть таблица `tags` — справочник тегов. По её образцу нужно создать остальные объекты. Скрипты из `db/init` PostgreSQL выполняет при первом создании базы в алфавитном порядке, поэтому сначала идёт схема, потом данные.

Имена таблиц и столбцов менять нельзя: на них опираются тесты и скрипт проверки.

Таблица	Столбец	Тип и ограничения
measurements	time	timestamptz, NOT NULL, по умолчанию now()
	tag_id	int, NOT NULL, ссылка на tags(id)
	value	double precision, NOT NULL
alarm_limits	tag_id	int, первичный ключ, ссылка на tags(id)
	lo_lo, lo, hi, hi_hi	double precision, могут быть NULL (уставка не задана)
	deadband	double precision, NOT NULL, по умолчанию 1.0
	alarm_value	double precision, может быть NULL (для дискретных тегов)
events	id	bigserial, первичный ключ
	time	timestamptz, NOT NULL, по умолчанию now()
	tag_id	int, NOT NULL, ссылка на tags(id)
	state	text, NOT NULL, одно из NORMAL, LO_LO, LO, HI, HI_HI, ALARM
	value	double precision, NOT NULL
	message	text, NOT NULL
	acked_at	timestamptz, может быть NULL
commands	id	bigserial, первичный ключ
	time	timestamptz, NOT NULL, по умолчанию now()
	tag_id	int, NOT NULL, ссылка на tags(id)
	value	double precision, NOT NULL
	source	text, NOT NULL

Справочник: SQL

Конструкция	Назначение и пример
`CREATE TABLE t (col type constraints, ...)`	создать таблицу: `CREATE TABLE pumps (id serial PRIMARY KEY, name text NOT NULL)`
`serial`, `bigserial`	целое с автоувеличением (4 и 8 байт)
`timestamptz`	дата и время с часовым поясом

Конструкция	Назначение и пример
`double precision`	вещественное число 8 байт
`DEFAULT expr`	значение по умолчанию: `created timestamptz NOT NULL DEFAULT now()`
`REFERENCES t(col)`	внешний ключ: `pump_id int NOT NULL REFERENCES pumps(id)`
`CHECK (условие)`	ограничение: `kind text CHECK (kind IN ('a', 'b'))`
`CREATE INDEX name ON t (col1, col2 DESC)`	индекс по нескольким столбцам
`CREATE VIEW v AS SELECT ...`	представление — сохранённый запрос
`SELECT DISTINCT ON (a) ... ORDER BY a, b DESC`	по одной строке на каждое значение a — первую в порядке сортировки
`INSERT INTO t (cols) SELECT ... FROM ...`	вставить строки, полученные запросом
`json_build_object('k1', v1, 'k2', v2)`	собрать JSON из пар ключ — значение
`pg_notify('канал', текст)`	отправить уведомление подписчикам канала

Пример: последнее показание каждого насоса.

```
SELECT DISTINCT ON (p.id) p.name, r.value, r.time
FROM pumps p JOIN readings r ON r.pump_id = p.id
ORDER BY p.id, r.time DESC;
```

Пример: триггерная функция, которая сообщает о каждой новой строке таблицы readings.

```
CREATE FUNCTION notify_reading() RETURNS trigger AS $$
BEGIN
    PERFORM pg_notify('readings', json_build_object('id', NEW.id, 'value',
NEW.value)::text);
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER readings_notify AFTER INSERT ON readings
FOR EACH ROW EXECUTE FUNCTION notify_reading();
```

Внутри триггерной функции `NEW` — вставляемая строка. `PERFORM` выполняет выражение и отбрасывает результат.

TODO 5. Таблицы

1) Создайте таблицы по спецификации выше.
2) Для measurements создайте индекс `measurements_tag_time_idx` по (tag_id, time DESC): так запрос «последние значения тега» читает индекс, а не всю таблицу.
3) Для events создайте индекс по time DESC: журнал обычно смотрят с конца.

TODO 6. Представление и уведомления

1) Создайте представление `current_values` со столбцами name, description, value, unit, time: для каждого тега его последнее измерение. Используйте `DISTINCT ON` и соединение tags с measurements.
2) Создайте функцию `notify_event()`, которая отправляет в канал `events` JSON с полями id, time, tag (имя тега, подзапросом к tags), state, value, message.

3) Создайте триггер `events_notify`, который вызывает её после каждой вставки в events.

TODO 7. Начальные данные

1) Добавьте в tags четыре тега по карте регистров (раздел 4.2):

name	description	unit	reg_type	address	scale
reactor_temp	Температура в реакторе	°C	input	0	0.1
heater_power	Мощность нагревателя (факт)	%	input	1	1.0
cooler_on	Охладитель включён		coil	0	1.0
gas_alarm	Датчик загазованности		discrete	0	1.0

2) Добавьте уставки: для reactor_temp — hi = 80, hi_hi = 90, deadband = 2; для gas_alarm — alarm_value = 1. Идентификаторы тегов не пишите числами, выбирайте по имени: `INSERT INTO alarm_limits (...) SELECT id, ... FROM tags WHERE name = '...'`.

Проверка

Запустите только базу и посмотрите, что получилось:

```
docker compose up -d db
psql -h localhost -U scada scada          # пароль: scada
scada=> \dt
scada=> \d events
scada=> SELECT t.name, a.* FROM alarm_limits a JOIN tags t ON t.id = a.tag_id;
```

Проверьте триггер. В одном окне psql подпишитесь на канал, во втором вставьте событие:

```
scada=> LISTEN events;
scada=> INSERT INTO events (tag_id, state, value, message) VALUES (1, 'HI', 81,
'проверка');
```

В окне с LISTEN после следующей команды (например, `SELECT 1;`) появится строка `Asynchronous notification "events" with payload "{...}"`. Удалите тестовую строку: `DELETE FROM events;`.

Скрипты из `db/init` выполняются только при первом создании базы. Если вы нашли ошибку и исправили SQL, базу нужно пересоздать вместе с томом данных:

```
docker compose down -v
docker compose up -d db
```

Ошибки в SQL видны в журнале контейнера: `docker compose logs db`.

Этап 3. Шлюз (TODO 8–11)

Что дано и что нужно сделать

В каталоге `gateway` даны модуль Go (`go.mod`, `go.sum` с зависимостями), файл `types.go` и тесты. В `types.go` описаны:

- `Tag` — тег: ID, имя, описание, единица, тип области Modbus (`input`, `holding`, `coil`, `discrete`, `calc`), адрес с нуля, масштаб и указатель на уставки `Limits` (nil, если уставок нет);
- `Sample` — пара «тег — значение»;
- `State` и константы `Normal`, `LoLo`, `Lo`, `Hi`, `HiHi`, `Alarm` — состояния аварии, строковые значения совпадают со столбцом events.state;
- `Limits` — уставки; nil в полях LoLo, Lo, Hi, HiHi, AlarmValue означает «не задана».

Порядок работы: драйвер ПЛК, хранилище, аварии, затем основной цикл, который их соединяет. После каждого файла запускайте его тесты.

Справочник: github.com/simonvetter/modbus

Функция / метод	Параметры	Возвращает
`modbus.NewClient(conf *modbus.ClientConfiguration)`	конфигурация: `URL` вида "tcp://host:port", `Timeout`	`*ModbusClient`, error
`(c) Open()`	—	error; устанавливает соединение
`(c) Close()`	—	error
`(c) ReadRegister(addr uint16, regType modbus.RegType)`	адрес с нуля; `modbus.HOLDING_REGISTER` или `modbus.INPUT_REGISTER`	uint16, error
`(c) ReadRegisters(addr, quantity uint16, regType)`	адрес, количество	[]uint16, error
`(c) ReadCoil(addr uint16)`	адрес	bool, error
`(c) ReadDiscreteInput(addr uint16)`	адрес	bool, error
`(c) WriteCoil(addr uint16, value bool)`	адрес, значение	error
`(c) WriteRegister(addr uint16, value uint16)`	адрес, значение	error
`(c) SetUnitId(id uint8)`	Unit ID (по умолчанию 1)	error

Пример: прочитать регистр хранения 4 и включить катушку 2.

```
client, err := modbus.NewClient(&modbus.ClientConfiguration{
    URL:     "tcp://192.168.0.10:502",
    Timeout: time.Second,
})
if err != nil {
    return err
}
if err := client.Open(); err != nil {
    return err
}
defer client.Close()

raw, err := client.ReadRegister(4, modbus.HOLDING_REGISTER)
if err != nil {
    return err
}
fmt.Println("знаковое значение:", int16(raw))
err = client.WriteCoil(2, true)
```

После ошибки обмена (таймаут, разрыв) соединение лучше закрыть и при следующем вызове открыть заново: так шлюз сам восстановится после перезапуска контроллера.

TODO 8. Драйвер ПЛК (plc.go)

1) Опишите в структуре `PLC` поля: адрес контроллера и указатель на `modbus.ModbusClient` (nil — соединения нет).

2) `NewPLC(url)` только запоминает адрес и не подключается. Подключение откладывается до первого чтения, поэтому шлюз можно запустить раньше контроллера.

3) Напишите вспомогательный метод `connect()`: если клиент уже есть, ничего не делать; иначе создать клиента с таймаутом 1 с, вызвать `Open()` и сохранить. И метод `reset()`: закрыть клиента и обнулить указатель.

4) `Read(t)`:

- вызовите `connect()`, при ошибке верните её;
- по `t.RegType` выберите вызов: для `input` и `holding` — `ReadRegister` с нужным типом и приведением результата к `int16`, затем к float64; для `coil` и `discrete` — `ReadCoil` или `ReadDiscreteInput`, true превращается в 1, false в 0;
- для неизвестного типа верните ошибку `fmt.Errorf("неизвестный тип регистра %q", t.RegType)`;
- если чтение вернуло ошибку, вызовите `reset()` и верните ошибку;
- верните значение, умноженное на `t.Scale`.

Тесты драйвера требуют работающий эмулятор:

```
docker compose up -d --build plc
cd gateway
MODBUS_URL=tcp://localhost:5020 go test -v -run PLC ./...
```

Справочник: github.com/jackc/pgx/v5

Функция / метод	Параметры	Возвращает
`pgxpool.New(ctx, connString)`	контекст; строка вида postgres://user:pass@host:5432/d	`*pgxpool.Pool`, error; соединения открываются по

Функция / метод	Параметры	Возвращает
	b	мере надобности
`(pool) Ping(ctx)`	контекст	error; проверяет, что база доступна
`(pool) Close()`	—	закрывает все соединения
`(pool) Exec(ctx, sql, args...)`	запрос с параметрами $1, $2, … и их значения	`pgconn.CommandTag`, error
`(pool) Query(ctx, sql, args...)`	запрос, параметры	`pgx.Rows`, error
`(rows) Next()` / `Scan(&a, &b, ...)` / `Err()` / `Close()`	переход к строке, чтение столбцов в переменные, ошибка перебора, освобождение	bool / error / error / —
`(pool) QueryRow(ctx, sql, args...).Scan(&a, ...)`	запрос, который возвращает одну строку	error (`pgx.ErrNoRows`, если строк нет)
`b := &pgx.Batch{}; b.Queue(sql, args...)`	добавить запрос в пакет	—
`(pool) SendBatch(ctx, b)`	пакет	`pgx.BatchResults`; вызов `Close()` у результата дожидается выполнения и возвращает первую ошибку

Столбец, который может быть NULL, читайте в указатель: `var hi *float64` и `Scan(&hi)`. После чтения hi равен nil, если в базе NULL. Именно так устроены поля `Limits`.

Пример: прочитать список насосов и добавить два показания одним пакетом.

```
pool, err := pgxpool.New(ctx, "postgres://u:p@localhost:5432/plant")
if err != nil {
    return err
}
defer pool.Close()

rows, err := pool.Query(ctx, `SELECT id, name, max_flow FROM pumps WHERE active =
$1`, true)
if err != nil {
    return err
}
defer rows.Close()
for rows.Next() {
    var id int
    var name string
    var maxFlow *float64 // может быть NULL
    if err := rows.Scan(&id, &name, &maxFlow); err != nil {
        return err
    }
}
if err := rows.Err(); err != nil {
    return err
}

b := &pgx.Batch{}
b.Queue(`INSERT INTO readings (pump_id, value) VALUES ($1, $2)`, 1, 12.5)
b.Queue(`INSERT INTO readings (pump_id, value) VALUES ($1, $2)`, 2, 7.0)
err = pool.SendBatch(ctx, b).Close()
```

Значения всегда передавайте параметрами, а не склейкой строк: иначе имя тега с кавычкой сломает запрос, а злоумышленник сможет выполнить свой SQL.

TODO 9. Хранилище (store.go)

1) В структуре `Store` храните `*pgxpool.Pool`.

2) `NewStore(ctx, url)`: создайте пул и проверьте связь через `Ping`. База в Docker может стартовать позже шлюза, поэтому при ошибке повторяйте Ping раз в секунду, до 30 попыток, и только потом возвращайте ошибку.

3) `Close()` закрывает пул.

4) `LoadTags(ctx)` читает все теги вместе с уставками одним запросом: `tags LEFT JOIN alarm_limits`, сортировка по id. Для каждой строки заполните `Tag`. Если у тега есть строка в alarm_limits (проверьте, например, `a.tag_id IS NOT NULL`), заполните `Limits` и сохраните указатель на них в `Tag.Limits`. NULL в deadband превратите в 0 через `COALESCE`. Не забудьте `rows.Err()`.

5) `SaveSamples(ctx, ts, samples)` записывает все значения опроса одним пакетом `pgx.Batch` с одинаковым временем ts. Пустой список не должен приводить к ошибке.

6) `SaveEvent(ctx, ts, t, st, v, msg)` добавляет строку в events. Состояние записывайте строкой: `string(st)`.

```
docker compose up -d db
cd gateway
DATABASE_URL=postgres://scada:scada@localhost:5432/scada go test -v -run 'Load|Save|
Current' ./...
```

Тесты создают временный тег, пишут по нему данные и удаляют за собой. Если тест упал на полпути, лишние строки останутся; проще всего пересоздать базу через `docker compose down -v`.

TODO 10. Аварии (alarms.go)

`Evaluate(prev, v, l)` получает предыдущее состояние, новое значение и уставки и возвращает новое состояние:

1) Если задан `AlarmValue` (дискретный тег): ALARM при `v == *AlarmValue`, иначе NORMAL. Остальные уставки не смотреть.

2) Проверки идут от более опасного к менее опасному: HH, H, LL, L. Первая сработавшая определяет результат. Если не сработала ни одна, результат NORMAL.

3) Верхняя уставка X срабатывает, если `v >= X`, или если тег уже был в этом либо более опасном состоянии и `v > X − Deadband`. Для H «более опасное» — это HH: при остывании из HH значение 87,5 должно дать HI, а не NORMAL.

4) Нижние уставки — зеркально: `v <= X` или (уже в этом либо более опасном состоянии и `v < X + Deadband`).

5) Уставка, равная nil, не проверяется. Разыменовывайте указатель только после проверки на nil.

`Message(desc, unit, s, v, l)` формирует текст события. Значение выводится с одним знаком после запятой и единицей, уставка — с одним знаком:

Состояние	Текст
HI_HI	Температура в реакторе: АВАРИЯ, превышен предел HH=90.0 (91.3 °C)
HI	Температура в реакторе: превышен предел H=80.0 (80.4 °C)
LO	…: ниже предела L=10.0 (9.8 °C)
LO_LO	…: АВАРИЯ, ниже предела LL=5.0 (4.1 °C)
ALARM	Датчик загазованности: АВАРИЯ
NORMAL	Температура в реакторе: возврат в норму (77.9 °C)

Для форматирования используйте `fmt.Sprintf("%s: превышен предел H=%.1f (%.1f %s)", desc, *l.Hi, v, unit)`. У дискретных тегов единицы нет; в этом случае лишнего пробела перед закрывающей скобкой быть не должно (`strings.TrimSpace` поможет), тест это проверяет.

```
cd gateway
go test -v -run 'Evaluate|Message' ./...
```

Прочитайте случаи в `TestEvaluate` и для каждого объясните, почему ожидается именно такой результат. Это пригодится на защите.

Справочник: стандартная библиотека Go для main

Функция	Назначение
`os.Getenv("NAME")`	значение переменной окружения или ""
`time.ParseDuration("1s")`	разобрать длительность ("500ms", "2s")
`signal.NotifyContext(ctx, os.Interrupt, syscall.SIGTERM)`	контекст, который отменяется по Ctrl+C или при остановке контейнера; возвращает ctx и функцию stop
`time.NewTicker(d)` / `ticker.C` / `ticker.Stop()`	канал, в который раз в d приходит отметка времени
`select { case <-ctx.Done(): ... case <-ticker.C: }`	ждать первого из событий
`log.Printf(...)`, `log.Fatalf(...)`	вывод в журнал; Fatalf завершает программу

Пример: цикл, который раз в секунду что-то делает и аккуратно завершается по сигналу.

```
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
defer stop()
ticker := time.NewTicker(time.Second)
defer ticker.Stop()
for {
    select {
    case <-ctx.Done():
        log.Print("остановка")
        return
    case <-ticker.C:
        log.Print("тик")
    }
}
```

TODO 11. Основной цикл (main.go)

1) Прочитайте настройки из окружения: `MODBUS_URL` (по умолчанию tcp://localhost:5020), `DATABASE_URL` (по умолчанию postgres://scada:scada@localhost:5432/scada), `POLL_INTERVAL` (по умолчанию 1s). Для значений по умолчанию удобно написать функцию `env(name, def string) string`.

2) Создайте контекст с остановкой по сигналу, хранилище (`NewStore`), загрузите теги (`LoadTags`), создайте драйвер (`NewPLC`). Ошибки на этом этапе фатальны: `log.Fatalf`.

3) Выведите строку `Шлюз запущен: ПЛК tcp://plc:5020, тегов 4, период опроса 1s`.

4) Заведите словарь текущих состояний аварий `map[int]State` (ключ — ID тега) и флаг «связь есть».

5) На каждом тике:

- прочитайте все теги, кроме `calc`, и соберите срез `Sample`. При первой же ошибке прекратите опрос в этом тике;
- если была ошибка, выведите «Нет связи с ПЛК: …» и переходите к следующему тику. Выводите сообщение один раз: при первой ошибке после запуска и при каждой потере связи, а не на каждом тике, иначе журнал утонет в одинаковых строках;
- при восстановлении связи выведите «Связь с ПЛК установлена»;
- запишите срез в архив (`SaveSamples`) с одним временем `time.Now()`. Ошибку записи выведите в журнал, но не останавливайте шлюз;
- для каждого значения с уставками вычислите новое состояние (`Evaluate`). Если оно отличается от прежнего (для нового тега прежнее — NORMAL), сформируйте текст (`Message`), выведите в журнал «СОБЫТИЕ HI: …» и сохраните (`SaveEvent`);
- по отмене контекста выведите «Остановка шлюза» и выйдите.

Проверьте шлюз без Docker. Запустите базу и эмулятор в контейнерах, а шлюз — из исходников:

```
docker compose up -d --build db plc
cd gateway
go run .
```

По умолчанию шлюз подключается к `localhost:5020` и `localhost:5432`, то есть к портам контейнеров. В журнале должно появиться примерно следующее (числа у вас будут другими):

```
Шлюз запущен: ПЛК tcp://localhost:5020, тегов 4, период опроса 1s
Связь с ПЛК установлена
СОБЫТИЕ HI: Температура в реакторе: превышен предел H=80.0 (80.3 °C)
СОБЫТИЕ HI_HI: Температура в реакторе: АВАРИЯ, превышен предел HH=90.0 (90.2 °C)
```

Посмотрите, что попало в базу:

```
psql -h localhost -U scada scada -c "SELECT * FROM current_values"
psql -h localhost -U scada scada -c "SELECT time, state, message FROM events ORDER BY
id"
```

Время в базе хранится с часовым поясом, а psql показывает его в поясе сервера, то есть в UTC. Не удивляйтесь разнице с часами на экране.

Остановите шлюз (Ctrl+C) и убедитесь, что он вывел «Остановка шлюза». Его нужно остановить, прежде чем переходить дальше, иначе в архив будут писать два шлюза сразу.

Этап 4. Запуск стенда и автоматическая проверка

Соберите и запустите все сервисы:

```
docker compose up -d --build
docker compose ps
```

Все четыре сервиса должны быть в состоянии running. Запустите проверку:

```
./scripts/check.sh
```

Скрипт проверяет контейнеры, наличие уставок, представления и триггера, поступление данных, формирование аварийного события и доступность FUXA. Если что-то не так, он остановится на первой ошибке и подскажет номер TODO. При правильном решении вывод выглядит так:

```
1. Контейнеры
  [OK]   plc запущен
  [OK]   db запущен
  [OK]   gateway запущен
  [OK]   fuxa запущен
2. Схема БД
  [OK]   тегов в справочнике: 4
  [OK]   уставок аварий: 2
  [OK]   представление current_values есть
  [OK]   триггер на events есть
3. Поступление данных
  [OK]   измерений за 10 с: 24
  [OK]   текущая температура: 96.9 °C
4. Аварийная сигнализация (ждём превышения H=80, до 180 с)
  [OK]   событий HI/HI_HI: 2
         15:43:30  HI  Температура в реакторе: превышен предел H=80.0 (83.8 °C)
         15:43:32  HI_HI  Температура в реакторе: АВАРИЯ, превышен предел HH=90.0
(92.9 °C)
5. FUXA
  [OK]   веб-интерфейс отвечает (HTTP 200)
Все проверки пройдены.
```

При PLC_SPEED = 1 четвёртый пункт ждёт около полутора минут: столько реактор греется до 80 °C.

Закоммитьте решение и отправьте его в свой репозиторий:

```
git add -A
git commit -m "Решение TODO 1-11"
git push
```

Теперь, при работающем стенде, прогоните все тесты шлюза, включая интеграционные:

```
cd gateway
MODBUS_URL=tcp://localhost:5020
DATABASE_URL=postgres://scada:scada@localhost:5432/scada go test -count=1 -v ./...
```

Все тесты должны быть PASS, ни одного SKIP. Тесты пишут в контроллер и базу, поэтому запускайте их после `check.sh`, а не одновременно с ним.

На вкладке Actions вашего репозитория запустится тот же набор проверок: сборка C++, `go vet`, unit-тесты, подъём стенда, `check.sh` и интеграционные тесты. Прогон занимает около четырёх минут. Зелёная отметка у коммита означает, что базовая часть сделана.

Этап 5Ф. Мнемосхема в FUXA

FUXA — SCADA-система с открытым исходным кодом (лицензия MIT), которая целиком работает в браузере: и редактор, и экран оператора. Она входит в состав стенда, отдельно ставить ничего не нужно. Все шаги ниже выполнены на FUXA 1.3.5 из стенда; в других версиях названия пунктов могут отличаться.

Первый запуск и язык

1) Откройте редактор: http://localhost:1881/editor. Появится окно «Initial Security Setup» с предложением включить аутентификацию. Для учебного стенда нажмите CLOSE. В реальной системе так делать нельзя, и в отчёте стоит объяснить почему.
2) Включите русский интерфейс: шестерёнка в левом верхнем углу → Settings → поле Language → Russian → OK. Дальше в тексте используются русские названия пунктов. Перевод местами неудачный («Зарегистрировать» вместо «Тип регистра»), поэтому в таких местах мы указываем, что имеется в виду.

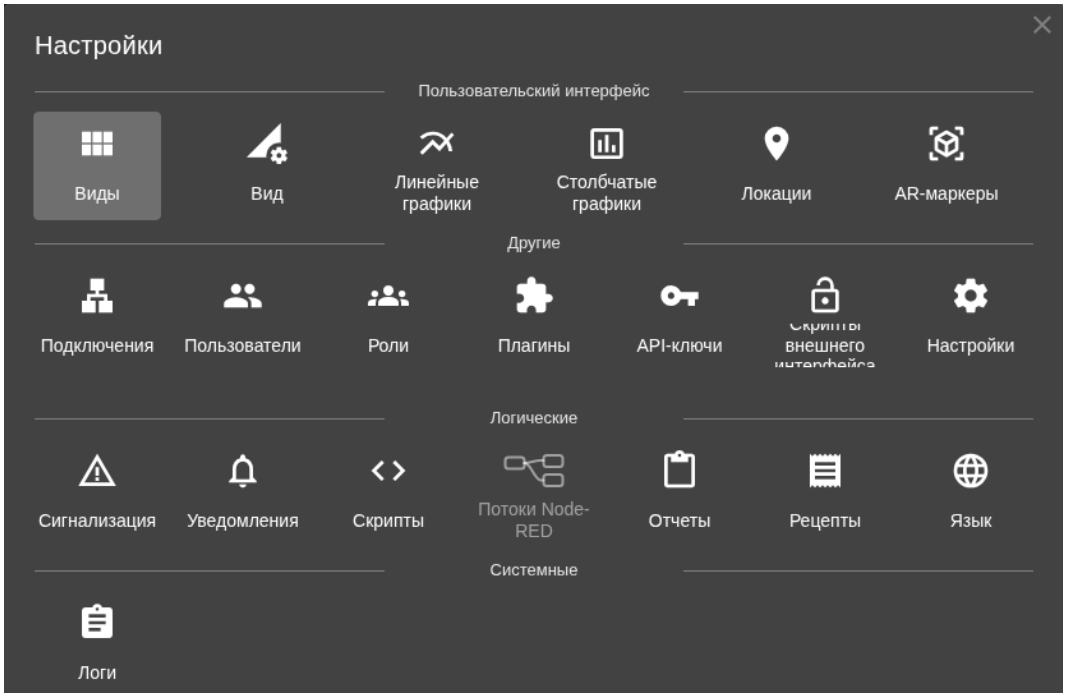


Рисунок 1 — Меню настроек FUXA

Драйвер Modbus

В образе FUXA нет драйвера Modbus: его ставят отдельно как плагин. Нужен доступ в интернет.

3) Шестерёнка → Плагины. Найдите карточку modbus-serial и нажмите Install. Через 10–30 секунд в поле «Установлен» появится версия 8.0.19.

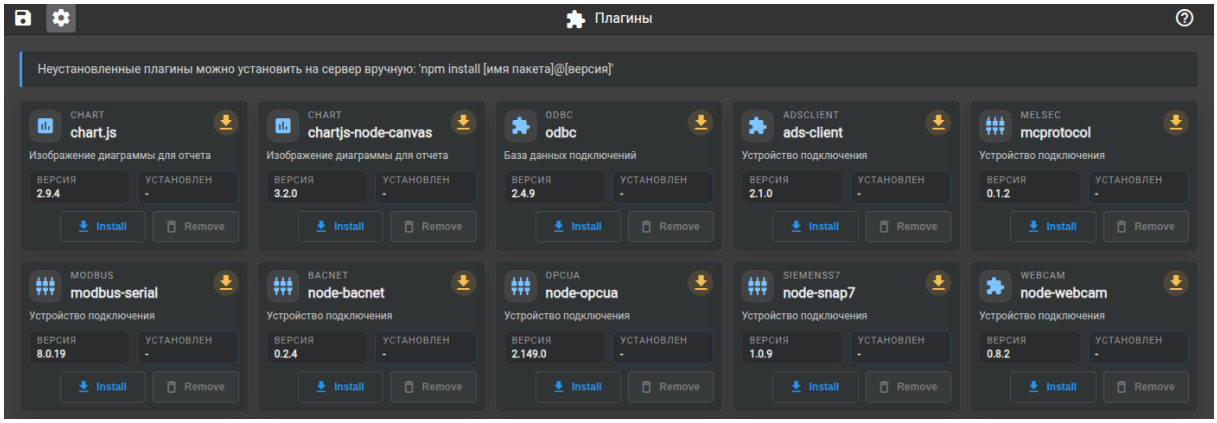


Рисунок 2 — Страница плагинов. Нужна карточка modbus-serial

Плагин сохраняется в томе Docker `fuxa_pkg`, поэтому после перезапуска контейнера ставить его снова не придётся. После `docker compose down -v` тома удаляются, и плагин нужно будет поставить заново.

Подключение к контроллеру

4) Шестерёнка → Подключения. Закройте подсказку о плагинах кнопкой «Закрыть». На экране одна карточка, FUXA Server. Нажмите круглую кнопку «+» в правом нижнем углу.

5) Заполните окно «Свойства устройства»:

Поле	**Значение**
Название	ПЛК
Тип	ModbusTCP
Опрос	1 sec
Включено	включить
Варианты подключения	TcpPort
Slave IP and Port	plc:5020
Идентификатор по… (Slave ID, поле справа от адреса)	1

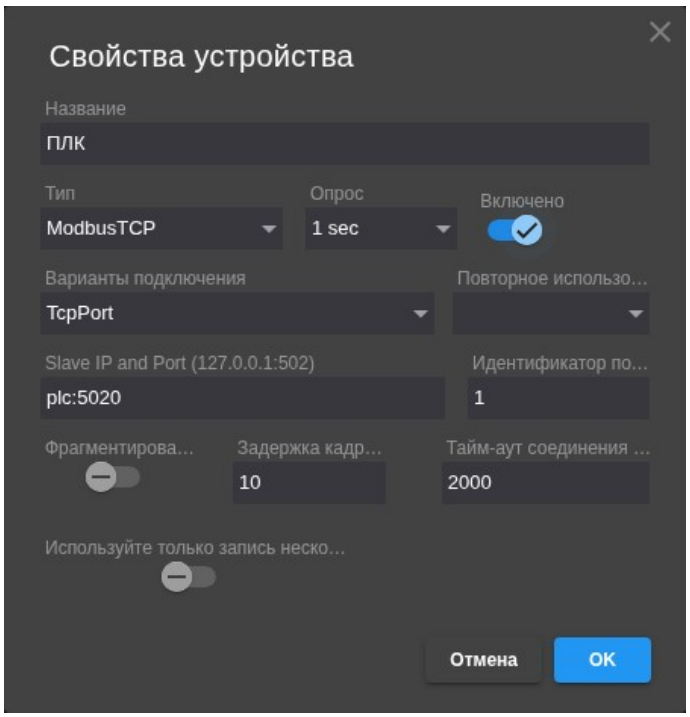


Рисунок 3 — Свойства устройства

Порт пишется в том же поле через двоеточие. Адрес `localhost` здесь не подойдёт: FUXA работает в своём контейнере, и для неё localhost — это сам контейнер FUXA, а не ваш компьютер. Нажмите OK. На карточке ПЛК должна загореться зелёная точка: связь есть.

Теги

6) На карточке ПЛК нажмите значок цепочки. Откроется пустой список тегов. Нажмите «+» в заголовке таблицы и заполните окно «Свойства тега». Поле «Зарегистрировать» на самом деле означает тип области Modbus, «Вид» — тип данных, «Смещение адреса» — номер регистра с единицы, «Делитель» — масштаб.

Имя тега	**Зарегистрировать (область)**	**Вид (тип)**	**Смещение адреса**	**Делитель**	**Описание**
temp	Input Registers	Int16	1	10	Температура, °C
heater	Input Registers	Int16	2	—	Мощность нагревателя, %
heater_sp	Holding Registers	Int16	1	—	Задание мощности, %
cooler	Coil Status	Bool	1	—	Охладитель
gas_leak	Coil Status	Bool	2	—	Имитация утечки
gas	Digital Inputs	Bool	1	—	Датчик газа

Для Coil Status и Digital Inputs тип Bool FUXA выставляет сама. Сразу после создания тега в столбце «Значение» должно появиться живое значение. Если

температура показывается как 384 вместо 38.4, вы забыли делитель; если значение не совпадает с mbpoll, проверьте номер регистра.

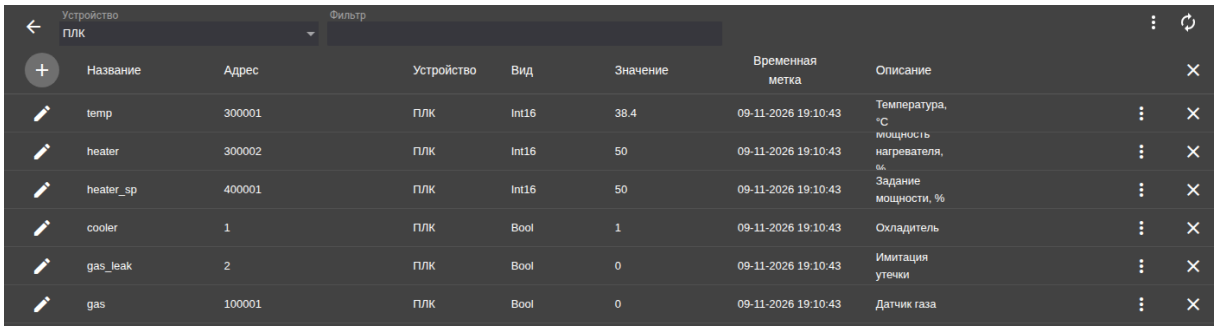


Рисунок 4 — Теги контроллера с текущими значениями

Экран оператора

7) Вернитесь в редактор (http://localhost:1881/editor). Слева находятся список видов (экранов) и палитра элементов. В проекте уже есть пустой вид MainView, на нём и будем рисовать. Если навести указатель на значок в палитре «Элементы управления», появится подсказка с названием элемента.

8) Выведите температуру. Щёлкните в палитре значок «##» (второй в первом ряду, подсказка «Выходное значение»), затем щёлкните на холсте: появится текст ##.##. Чтобы настроить элемент, выделите его одним щелчком и в правой панели в разделе «Интерактивность» нажмите кнопку «Настройка».

Двойной щелчок по элементу открывает не настройки, а режим редактирования внутренних частей группы: вверху холста появляется строка вида «Layer 1 > VAL_...». Выйти из этого режима можно, щёлкнув «Layer 1».

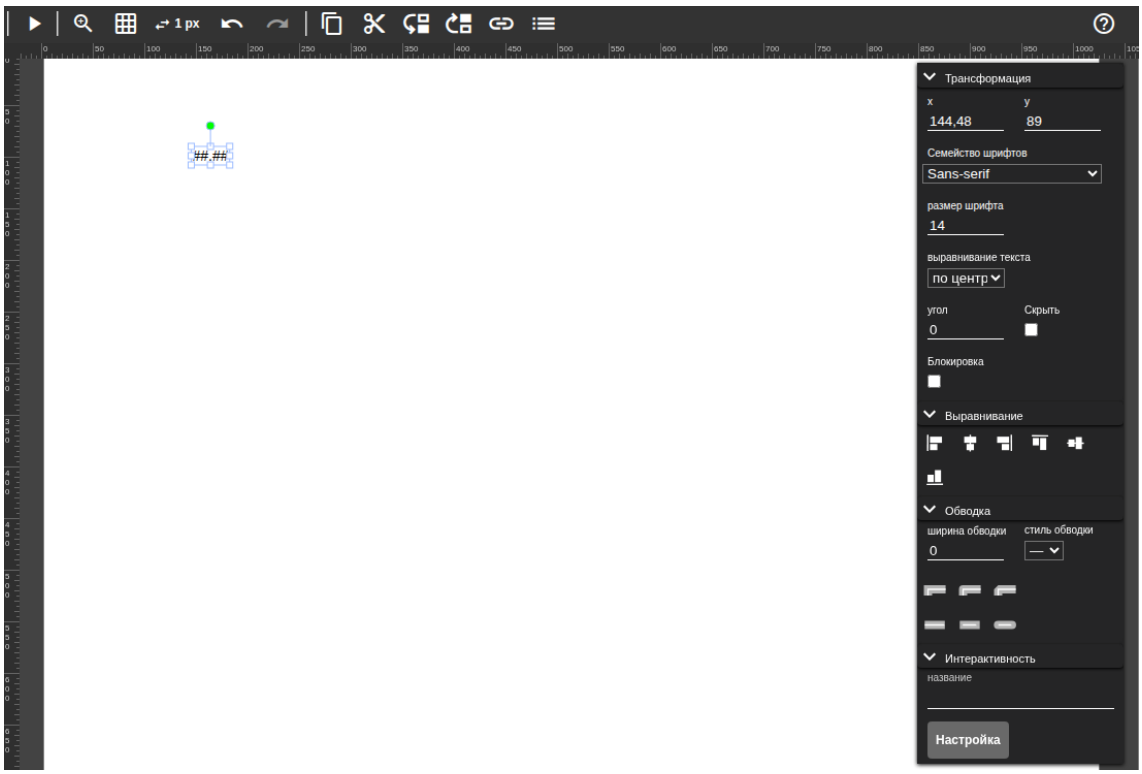


Рисунок 5 — Выделенный элемент и кнопка «Настройка» в правой панели

9) В окне «Настройки выходного значения» нажмите значок цепочки рядом с полем «Тег», в списке отметьте temp и нажмите OK, затем ещё раз OK.

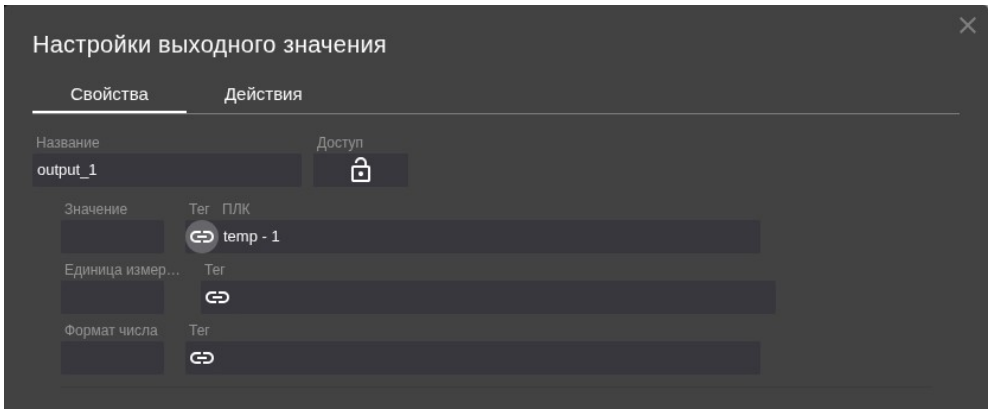


Рисунок 6 — Привязка элемента к тегу temp

10) Добавьте кнопку управления охладителем. Значок кнопки — третий в первом ряду палитры («Кнопка»). Поставьте кнопку на холст, выделите её и нажмите «Настройка». На вкладке «Свойства» в поле «Текст» напишите ВКЛ. На вкладке «События» выберите: Вид — Клик, Действие — Установка значения, Значение — 1, Тег — cooler. Сделайте вторую кнопку, ВЫКЛ, со значением 0.

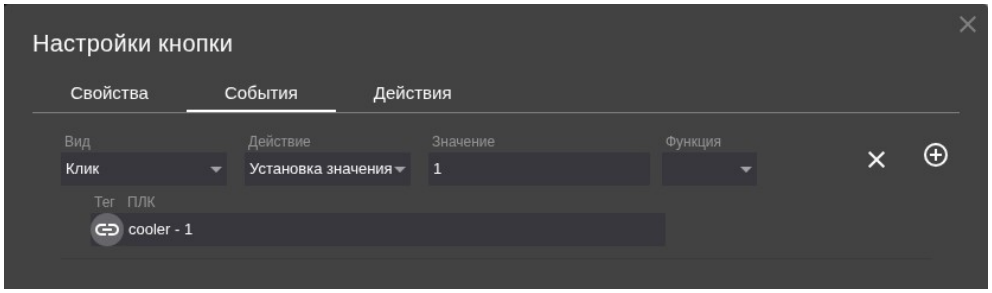


Рисунок 7 — Событие кнопки: по щелчку записать 1 в тег cooler

11) Самостоятельно добавьте на экран:

- значение мощности нагревателя (тег heater) и подписи с единицами измерения (инструмент «Текст» в палитре «Общие»);
- поле ввода задания мощности (элемент «Входное значение», первый значок палитры, тег heater_sp). На работающем экране введите число и нажмите Enter, затем проверьте через mbpoll, что регистр 400001 изменился;
- цветные индикаторы состояния охладителя и датчика газа (значок в виде лампочки; в русском переводе подсказка неудачная — «Панель приборов»). В его настройках задаются диапазоны значений и цвет для каждого;
- кнопки «Утечка» и «Сброс» для катушки gas_leak, чтобы проверить аварию по газу.

12) Сохраните проект: значок дискеты → «Сохранить проект». Сам по себе значок только открывает меню. Если закрыть или обновить страницу, не сохранив проект, нарисованные элементы пропадут.

Стартовый экран и индикатор аварий

13) Шестерёнка → Вид. На вкладке «Общие» проверьте, что стартовая страница — MainView. На вкладке «Панель навигации в заголовке» в поле «Режим отображения Сигнализации» выберите «Фиксированный»: в заголовке экрана оператора появится колокольчик со счётчиком активных аварий. Нажмите OK и снова сохраните проект.

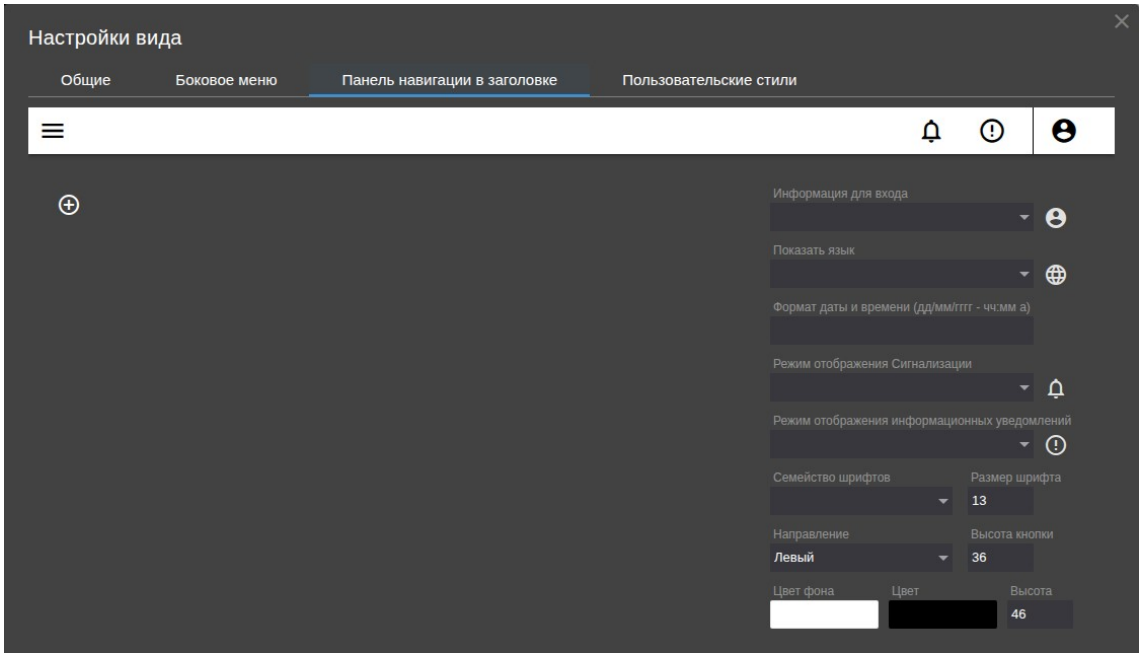


Рисунок 8 — Настройка заголовка экрана оператора

Аварии

14) Шестерёнка → Сигнализация → «+». В окне «Оповещение» задайте название «Температура» и выберите тег temp. На вкладке «Очень высокая» включите переключатель «Включено» и задайте: интервал проверки 1, Мин 90, Макс 10000, время задержки 1, текст «АВАРИЯ: температура выше 90 °C». На вкладке «Высокая» — то же самое с границами 80 и 90 и текстом-предупреждением. Нажмите OK.

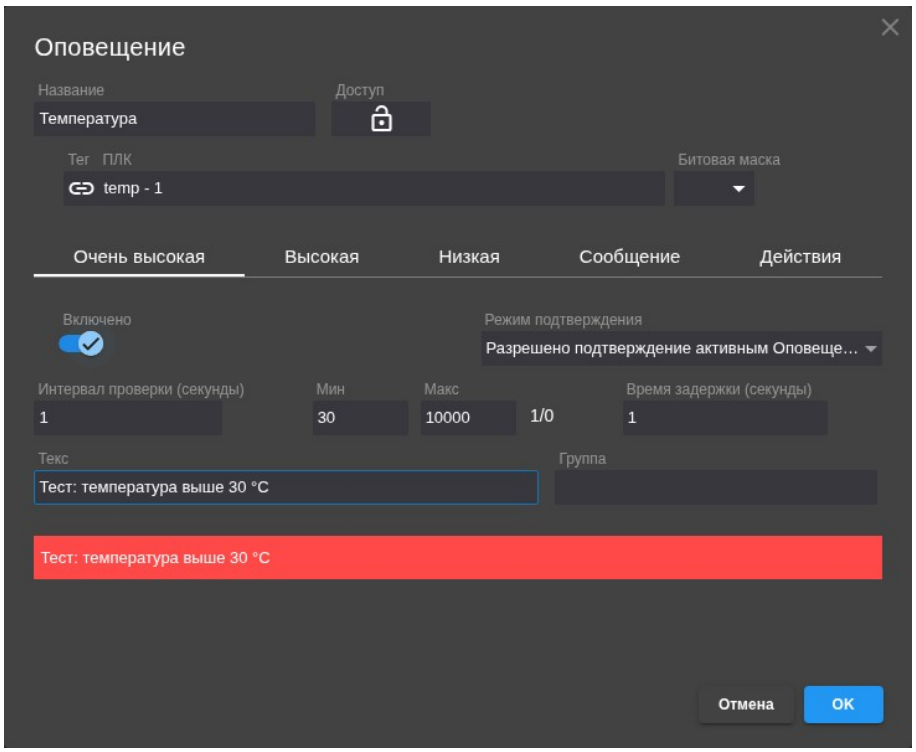


Рисунок 9 — Окно настройки аварии. На снимке тестовая уставка 30 °C, у вас должно быть 90

В FUXA авария активна, пока значение лежит в интервале от «Мин» до «Макс». Отдельной зоны возврата, как в шлюзе, здесь нет, есть только задержка срабатывания. Сравните поведение двух систем около уставки и опишите разницу в отчёте.

15) Добавьте вторую аварию: тег gas, вкладка «Очень высокая», Мин 1, Макс 1, текст «АВАРИЯ: загазованность».

Проверка

16) Откройте экран оператора: http://localhost:1881. Убедитесь, что температура меняется, кнопки ВКЛ и ВЫКЛ включают и выключают охладитель (проверьте через mbpoll или по индикатору), а при превышении 80 °C на колокольчике появляется счётчик. Щелчок по колокольчику открывает панель «Оповещения», где аварию можно квитировать.

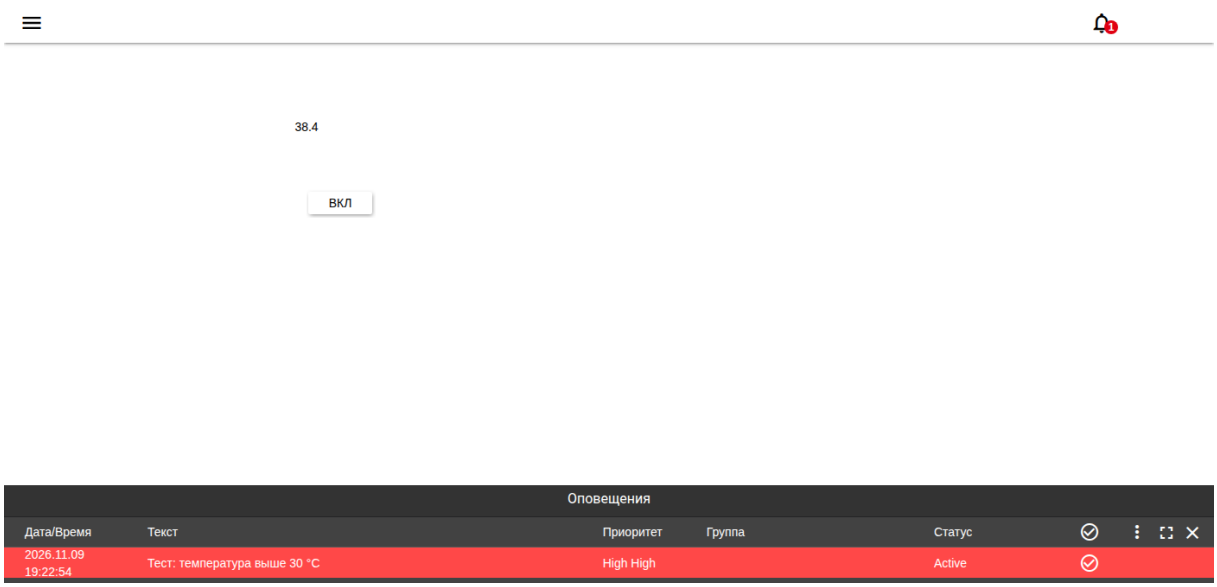


Рисунок 10 — Экран оператора: счётчик на колокольчике и панель «Оповещения»

Для сравнения ниже показан вариант мнемосхемы, который сделал преподаватель. Повторять его в точности не нужно, но все перечисленные элементы на вашем экране должны быть.

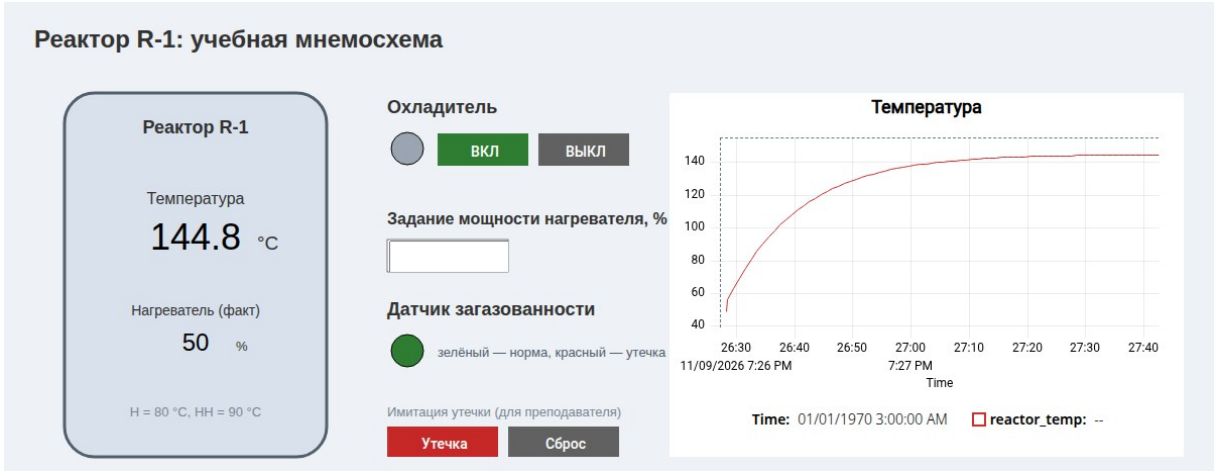


Рисунок 11 — Пример готовой мнемосхемы (PLC_SPEED = 10, охладитель выключен)

17) Сохраните конфигурацию в репозиторий. FUXA отдаёт весь проект в формате JSON по адресу /api/project:

```
mkdir -p fuxa
curl -s http://localhost:1881/api/project > fuxa/project.json
git add fuxa/project.json
git commit -m "Проект FUXA"
```

Этап 5M. MasterSCADA 4D (в учебном кабинете)

> Этот раздел составлен по документации MasterSCADA 4D и ещё не проверен на установке в кабинете. Названия пунктов меню могут отличаться. Если вы нашли расхождение, сообщите преподавателю.

MasterSCADA 4D — российская SCADA-система компании ИнСАТ. Лицензия есть только на компьютерах учебного кабинета, поэтому этот вариант

этапа 5 выполняется на занятии. Общая часть стенда (эмулятор, база, шлюз) остаётся прежней: она работает на Linux-компьютере или в виртуальной машине, а MasterSCADA — на Windows-компьютере кабинета и подключается к эмулятору по сети.

Подготовка:

1) На компьютере со стендом узнайте IP-адрес (`ip -br addr`) и убедитесь, что порт 5020 доступен с компьютера с MasterSCADA. Если на Linux включён межсетевой экран, откройте порт: `sudo ufw allow 5020/tcp`.
2) На Windows-компьютере проверьте связь: `Test-NetConnection <IP> -Port 5020` в PowerShell. Должно быть TcpTestSucceeded : True.

Порядок работы в среде разработки:

3) Создайте новый проект. В дереве системы добавьте узел среды исполнения (компьютер), если его нет.
4) Добавьте к узлу драйвер Modbus TCP (ведущий, master) и в его параметрах укажите IP-адрес компьютера со стендом, порт 5020 и адрес устройства 1.
5) Создайте переменные по карте регистров из раздела 4.2: temp, heater, heater_sp, cooler, gas_leak, gas. Узнайте в документации, с какого номера MasterSCADA считает адреса, с нуля или с единицы, и сверьте значения с mbpoll. Для температуры настройте масштабирование 0,1 или деление на 10.
6) Нарисуйте мнемосхему с тем же составом элементов, что в шагах 8–11 варианта FUXA: значения, кнопки управления охладителем, поле ввода задания, индикаторы.
7) Настройте аварийные границы для температуры (80 и 90 °C) и аварию по датчику газа. Найдите в настройках параметр гистерезиса и задайте 2 °C.
8) Запустите проект в режиме исполнения и проверьте то же, что в шаге 16.

Учтите ограничения бесплатных версий. По сведениям из открытых источников, демонстрационная среда исполнения перезапускается примерно через час работы, а лицензия на 32 точки не позволяет архивировать данные во внешнюю базу. Внешний архив в нашем стенде ведёт шлюз, поэтому для работы это не критично.

Этап 6. Эксперименты

Выполните опыты и запишите результаты в отчёт. Для наглядности графиков поставьте в `.env` значение `PLC_SPEED=1` и перезапустите стенд (`docker compose up -d`).

1) Установившаяся температура. Дождитесь, пока температура перестанет расти, при выключенном и при включённом охладителе. Сравните с расчётом из раздела 3.6 (145 °C и 38,4 °C). Найдите по архиву время

достижения 80 °C и сравните его с аналитическим решением T(t) = 145 − 125·e^(−t/125).

2) Гистерезис. Подберите задание мощности так, чтобы температура установилась чуть выше 80 °C (P = 25 %, расчётное значение 82,5 °C), и включите шум измерения: `mbpoll -m tcp -p 5020 -t 4 -r 2 -1 localhost 20` (амплитуда 2,0 °C). Сравните, сколько событий за 5 минут записал шлюз (с зоной возврата 2 °C) и сколько раз срабатывала авария в FUXA. Объясните разницу.

3) Потеря связи. Остановите контроллер (`docker compose stop plc`), подождите 10 секунд и запустите снова (`docker compose start plc`). Посмотрите журнал шлюза (`docker compose logs gateway --tail 20`): в нём будут строки «Нет связи с ПЛК: EOF» и «Связь с ПЛК установлена». Что в это время показывала FUXA? Почему после перезапуска температура снова начала расти с 20 °C и чем в этом отношении настоящий ПЛК отличается от эмулятора?

4) Запись без авторизации. При работающем стенде выполните с компьютера, а не из SCADA, команду `mbpoll -m tcp -p 5020 -t 4 -r 1 -1 localhost 100`. Что произошло с реактором? Отразилась ли эта команда в таблице commands? Есть ли у SCADA способ отличить такую запись от команды оператора? Предложите меры защиты.

6. Дополнительные задания

Задания выполняются по указанию преподавателя или для повышения оценки. Все они реализованы и проверены в эталонном решении; там, где это важно, указан результат, которого стоит ожидать.

6.1. Автоматический регулятор охладителя

Добавьте в шлюз двухпозиционный регулятор с гистерезисом: охладитель включается при T ≥ CONTROL_ON и выключается при T ≤ CONTROL_OFF. Пороги берите из переменных окружения; если они не заданы, регулятор выключен. Каждую команду записывайте в таблицу commands с источником gateway.

Проверьте регулятор с порогами 85 и 70, затем 75 и 60. В первом случае предупреждение H неизбежно: температура проходит 80 °C раньше, чем регулятор вообще начинает действовать. Порог включения должен быть ниже уставки аварии, но насколько ниже? Учтите, что шлюз видит температуру раз в секунду, а при PLC_SPEED = 10 за это время проходит 10 модельных секунд. Оцените по уравнению модели, на сколько градусов температура успеет подняться между двумя опросами вблизи 75 °C, и сравните с тем, что видно в архиве. Для сравнения: в эталонном решении при PLC_SPEED = 10 и порогах 75/60 температура доходила

до 80,4 °C и предупреждение H несколько раз всё же срабатывало. Проверьте то же при PLC_SPEED = 1.

Нажмите на мнемосхеме ВЫКЛ, когда регулятор держит охладитель включённым. Регулятор на следующем же цикле включит его обратно. Предложите, как организовать режимы «ручной» и «автомат» и где хранить текущий режим.

6.2. Тренды и архив в Grafana

Добавьте в docker-compose.yml сервис Grafana (образ grafana/grafana-oss:12.4.3) и подключите PostgreSQL как источник данных. Постройте дашборд: текущая температура, график за последние 15 минут, средняя и максимальная температура по минутам (функция `date_bin`), таблица событий. Источник данных и дашборд лучше описать файлами provisioning, чтобы они создавались при запуске автоматически.



Рисунок 12 — Дашборд Grafana в эталонном решении

6.3. Уведомления через LISTEN/NOTIFY

Триггер, который вы написали в TODO 6, отправляет каждое новое событие в канал events. Напишите на Go отдельный сервис, который подписывается на канал (`LISTEN events`, функция `WaitForNotification` в pgx), выводит событие в журнал и отправляет его в Telegram-бот или на вебхук. Измерьте задержку от записи события до его получения. В эталонном решении она составляет около 10

мс (в наших прогонах от 5 до 12 мс). Объясните, почему такой подход лучше, чем раз в секунду опрашивать таблицу events.

6.4. Резервирование шлюза

Запустите два экземпляра шлюза так, чтобы опрашивал контроллер только один, а второй ждал в резерве. Для выбора ведущего используйте рекомендательную блокировку PostgreSQL: `pg_try_advisory_lock(id)`. Блокировка живёт, пока открыто соединение, в котором её взяли, поэтому это соединение нужно держать отдельно и следить за ним. Остановите ведущий экземпляр (`docker compose stop gateway`) и по архиву определите, сколько данных потерялось при переключении. В эталонном решении резервный экземпляр становится ведущим примерно через секунду; в архиве один интервал между опросами вырастает до 2 с, то есть при переключении теряется не больше одного опроса. Подумайте, как сократить и эту потерю.

6.5. Фильтрация измерений

Включите шум измерения (Holding 1 = 20) и добавьте в шлюз расчётный тег reactor_temp_ema — экспоненциальное скользящее среднее: $y(k) = \alpha \cdot x(k) + (1 - \alpha) \cdot y(k-1)$. Тег нужно добавить в справочник tags с типом calc. Постройте на одном графике сырой и сглаженный сигнал при $\alpha = 0{,}2$ и $\alpha = 0{,}05$. Оцените, насколько фильтр запаздывает и как это влияет на момент срабатывания аварии, если вычислять её по сглаженному значению.

7. Частые проблемы

Симптом	Причина и решение
permission denied while trying to connect to the Docker daemon	пользователь не в группе docker: `sudo usermod -aG docker $USER`, затем перелогиниться
контейнер db не стартует, в логе address already in use	на компьютере уже работает PostgreSQL. Остановите его (`sudo systemctl stop postgresql`) или поменяйте PG_PORT в .env
контейнер plc не стартует, порт 5020 занят	не остановлен эмулятор, запущенный вручную на этапе 1
исправления в db/init не применяются	скрипты инициализации выполняются один раз. Нужно `docker compose down -v`
образы скачиваются очень долго	Docker Hub бывает медленным. Скачайте образы заранее (`docker compose pull`) или попросите у преподавателя архив образов
в FUXA нет типа ModbusTCP	не установлен плагин modbus-serial
на карточке ПЛК в FUXA нет зелёной точки	в адресе указан localhost вместо plc:5020, или не включён переключатель «Включено»
значение в FUXA сдвинуто на один регистр	в FUXA номера с единицы, а в коде адреса с нуля (раздел 3.3)
температура 384 вместо 38.4	не задан делитель 10 у тега temp
нарисованные элементы пропали	проект не сохранён через «Сохранить проект»
check.sh не дождался аварии	шлюз не пишет события (TODO 9, 11), не загружены уставки (TODO 7), неверна модель

Симптом	Причина и решение
	(TODO 2) или реактор охлаждается: проверьте Coil 0
тест падает с panic: TODO N	функция из TODO N ещё не написана
интеграционные тесты пишут SKIP	не заданы MODBUS_URL и DATABASE_URL
контейнер plc в состоянии restarting	эмулятор завершается сразу после запуска (TODO 4); смотрите docker compose logs plc
время в psql отличается на 3 часа	psql показывает время в UTC, это нормально

8. Сдача работы

Работа сдаётся в два шага.

1) Репозиторий. В вашем приватном репозитории должны лежать решения TODO 1–11, экспорт проекта FUXA в каталоге `fuxa/` и, если вы их делали, дополнительные задания. Последний коммит должен быть с зелёной отметкой CI. Ссылку на репозиторий отправьте преподавателю. Если вы работали без GitHub, покажите на занятии работающий стенд и вывод `check.sh`.

2) Отчёт в формате PDF. Он должен содержать:

- титульный лист;
- цель работы;
- схему стенда своими словами: какие компоненты, кто кого опрашивает, по какому протоколу и порту;
- фрагменты своего кода для TODO 1–11 с короткими пояснениями (не весь файл);
- вывод `check.sh` и `go test -v` с интеграционными тестами;
- снимки экрана: список тегов FUXA, мнемосхема в режиме исполнения, панель аварий;
- результаты экспериментов этапа 6 с графиками или таблицами и выводами;
- описание дополнительных заданий, если они выполнялись;
- ответы на контрольные вопросы;
- общий вывод: что получилось, что нет и почему.

9. Контрольные вопросы

1) Чем отличаются задачи ПЛК и SCADA? Что произойдёт с технологическим процессом, если сервер SCADA выключится?

2) Почему SCADA относят к системам мягкого реального времени? Приведите пример задачи, которую нельзя возлагать на SCADA.

3) Перечислите четыре области данных Modbus. Почему нельзя записать значение во входной регистр?

4) Регистр 300001 и адрес 0 в libmodbus — это один и тот же регистр? Объясните.

5) Как передать по Modbus температуру −12,7 °C? А давление 1013,25 гПа с точностью до сотых?

6) Зачем нужен гистерезис в аварийной сигнализации? Что будет, если выбрать зону возврата слишком большой?

7) Почему в TODO 2 нужно приводить значение сначала к int16_t, а потом к uint16_t? А почему в драйвере на Go (TODO 8) результат ReadRegister приводят к int16?

8) Какую выгоду даёт pgx.Batch по сравнению с отдельными INSERT? Почему значения нельзя подставлять в SQL склейкой строк?

9) Почему внутри Docker шлюз обращается к контроллеру по адресу plc:5020, а mbpoll с вашего компьютера — по localhost:5020?

10) Как изменится точность модели при PLC_SPEED = 100? При каком шаге h метод Эйлера для нашей модели станет неустойчивым?

11) Какие угрозы несёт отсутствие аутентификации в Modbus? Какие меры защиты применяют на практике?

12) Почему FUXA и шлюз опрашивают контроллер независимо? Какие у такой схемы плюсы и минусы?

13) Зачем в serve нужен таймаут select()? Что произойдёт без него при нажатии Ctrl+C?

14) Почему NewPLC не подключается к контроллеру сразу, а NewStore повторяет Ping до 30 раз?

10. Литература и ссылки

1) Modbus Application Protocol Specification V1.1b3. — Modbus Organization, 2012. — https://modbus.org/specs.php

2) Документация libmodbus. — https://libmodbus.org/reference/

3) Пакет github.com/simonvetter/modbus. — https://pkg.go.dev/github.com/simonvetter/modbus

4) Пакет github.com/jackc/pgx/v5. — https://pkg.go.dev/github.com/jackc/pgx/v5

5) Документация PostgreSQL 17: SELECT DISTINCT ON, LISTEN/NOTIFY, рекомендательные блокировки. — https://www.postgresql.org/docs/17/

6) FUXA: исходный код и вики. — https://github.com/frangoteam/FUXA

7) MasterSCADA 4D: документация. — https://masterscada.ru

8) Docker Compose: справочник. — https://docs.docker.com/compose/

Приложение А. Памятка по командам

Действие	Команда
запустить стенд	docker compose up -d --build
состояние сервисов	docker compose ps
журнал сервиса	docker compose logs -f gateway
остановить, сохранив данные	docker compose down
остановить и удалить данные	docker compose down -v
перезапустить сервис	docker compose restart gateway
проверка решения	./scripts/check.sh
тесты шлюза	cd gateway && go test -v ./...
прочитать входные регистры 1–2	mbpoll -m tcp -p 5020 -t 3 -r 1 -c 2 -1 localhost
записать задание мощности	mbpoll -m tcp -p 5020 -t 4 -r 1 -1 localhost 60
включить охладитель	mbpoll -m tcp -p 5020 -t 0 -r 1 -1 localhost 1
консоль базы	psql -h localhost -U scada scada
текущие значения	SELECT * FROM current_values;
последние события	SELECT time, state, message FROM events ORDER BY id DESC LIMIT 10;